

Flusso dei messaggi asincroni — PrinterServer

Ultima revisione: 2026-05-07 — Documento aggiornato dopo le sessioni di bugfix. Descrive il flusso **attuale** del codice, incluse tutte le modifiche apportate.

1. Schema generale del flusso (stato attuale)



2. Avvio: connessione e sync iniziale



3. handleIncomingData(data) — dispatcher.ts (messaggio STOMP in arrivo)

```

handleIncomingData(data)
  |
  |— GUARD tipo non in {PKMI_UPDATE, PKMI_ADD_ALL, PKMI_ADD} → return (warn)
  |— GUARD !data.plateKitchenMenuItem || !menuItem → return
(silenzioso)
  |
  |— if PKMI_UPDATE && status in {PROGRESS, DONE}
  |   |— scheduleResync() ← debounce 3000ms
  |
  |— switch(status):
  |   |— TODO:
  |   |   |— plate==null → return (warn)
  |   |   |— plate!=null → saveTodoPlate(id, plate.name) ← FIX #3 — DB
  |   |— persistente
  |   |   |— PROGRESS:
  |   |   |   |— plate==null → return (warn)
  |   |   |   |— DONE: → log + return (non stampato)
  |   |   |   |— CANCELLED: → log + return (non stampato)
  |   |— Risolve plate per PROGRESS: ← FIX #3
  |   |   |— getTodoPlate(id) → se plate originale diversa → usa plate originale
  |   |— Costruisce OrderPayload (con plate risolta)
  |   |— await handleIncomingOrder(order)

```

4. handleSingleOrderData(items[]) — dispatcher.ts (sync/resync)

```

handleSingleOrderData(items[])
  |
  |— for each item (seriale, for-of await):
  |   |— if TODO && plate.name → saveTodoPlate(id, plate) ← FIX #3
  |   |— if PROGRESS:
  |   |   |— getTodoPlate(id)
  |   |   |   |— plate diversa → usa plate originale (log) ← FIX #3
  |   |— await handleIncomingOrder(order)

```

5. handleIncomingOrder(order) — dispatcher.ts (stampa)

```

handleIncomingOrder(order)
  |
  |— sendNotification('NEW_TICKETS', ...) ← WebSocket push UI
  |— groupBy(items, dest)

```

```

└─ for each [dest, items]:
    └─ if status == TODO || DONE → warn + continue      ← FIX #5 (check
prima)
    └─ find printer for dest
        not found → logger.warn VISIBILE + continue    ← FIX #5
    └─ await enqueuePrinterJob(dest, async () => {
        └─ getReceiptByIdAndStatus(orderId, status)
            già presente → return (idempotency)
        └─ buildKitchenTicket_v2(...)
        └─ if printer.active → await sendToPrinter(ip, port, buffer)
        └─ saveReceipt(...) ← sincrono bun:sqlite
        └─ sendNotification('RECEIPT_PRINTED' |
'RECEIPT_PRINT_FAILED')
    })

└─ if status != TODO → deleteTodoPlate(orderId) ← FIX #3b (non cancella su
TODO)

```

6. Resync debounced (3s dopo PKMI_UPDATE PROGRESS/DONE)

```

scheduleResync() → setTimeout 3000ms (reset ad ogni evento)
└─ resyncCallback():
    └─ fetchItems(TODO, PROGRESS) → items[]
        processedIds = items.map(i => i.id)
    └─ handleSingleOrderData(items) ← stampa ordini PROGRESS
trovati
    └─ reconcileMissedOrders(processedIds) ← FIX #4

reconcileMissedOrders(processedIds):
└─ getAllTodoPlateCache() ← tutti gli ordini TODO ancora in attesa nel DB
└─ filtra: entry NOT IN processedIds ← ordini "scomparsi" dal fetchItems
└─ Promise.allSettled( fetchItemById per ognuno ) ← parallelo
    └─ CANCELLED → deleteTodoPlate
    └─ not found → deleteTodoPlate (warn)
    └─ DONE + mai stampato (no receipt PROGRESS) →
        △ STAMPA TARDIVA con plate originale dalla cache
        → handleIncomingOrder(order forzato a PROGRESS)
    └─ DONE + già stampato → deleteTodoPlate
    └─ TODO ancora (edge case paginazione) → mantenuto in cache

```

7. todo_plate_cache — DB persistente (nuova tabella)

```
CREATE TABLE todo_plate_cache (
  orderId TEXT PRIMARY KEY,
  plateName TEXT NOT NULL,      -- plate di nascita, immutabile (INSERT OR
  IGNORE)
  createdAt TEXT NOT NULL
);
```

Operazione	Quando	Metodo DB
Scrittura	Primo avvistamento TODO (STOMP o sync)	<code>saveTodoPlate</code> — INSERT OR IGNORE
Lettura	Risoluzione plate per ordini PROGRESS	<code>getTodoPlate</code>
Cancellazione	Dopo stampa/idempotency (non su TODO)	<code>deleteTodoPlate</code>
Lettura lista	Riconciliazione post-resync	<code>getAllTodoPlateCache</code>

Scenario crash/reboot coperto:

```
t=0   PKMI_ADD: X TODO plate=PANINI → DB: {X: "PANINI"}
t=5s  CRASH
t=10s Riavvio + sync iniziale: X già PROGRESS plate=PASS
      getTodoPlate("X") → "PANINI" ← sopravvissuto al crash
      → stampa su PANINI ✓
```

8. Legenda stati ordine

Status	handleIncomingData	handleIncomingOrder	Effetto finale
TODO	salva plate nel DB	skip stampa (warn)	solo plate memorizzata
PROGRESS	risolve plate originale	stampa	ticket prodotta
DONE	ignorato (log)	skip (warn)	nessuna azione
CANCELLED	ignorato (log)	skip (warn)	nessuna azione

9. Bug risolti (riepilogo cronologico)

#	Problema	Causa radice	Fix applicato
1	Perdita eventi in burst	Subscriber STOMP sincrono, Promise non awaited né catchata	<code>_pkmiQueue</code> Promise chain seriale in <code>WSClientController</code> ; JSON.parse in try-catch

#	Problema	Causa radice	Fix applicato
2	TypeError silenzioso su DONE/CANCELLED	<code>plate.name</code> senza optional chaining → rigetto Promise silenziosa	<code>plate?.name</code> in 3 punti di <code>dispatcher.ts</code>
3	Stampa sulla stampante sbagliata post-resync	La plate cambia tra TODO e PROGRESS; il resync vede già quella nuova	Tabella <code>todo_plate_cache</code> nel DB (persistente a crash); <code>saveTodoPlate/getTodoPlate</code>
3b	Cache svuotata subito dopo scrittura	<code>deleteTodoPlate</code> chiamata anche per ordini TODO	<code>if (status !== "TODO")</code> prima della delete
4	Ordini TODO→PROGRESS→DONE persi nel debounce	<code>fetchItems</code> non restituisce DONE; nessun meccanismo di recovery	<code>reconcileMissedOrders</code> + <code>setFetchItemCallback</code> ; fetch parallelo via <code>Promise.allSettled</code>
5	Warning stampante non trovata mai visibile	Check status/printer nell'ordine sbagliato; warn commentato	Status check PRIMA della ricerca stampante; warn sempre visibile per PROGRESS

10. Componenti coinvolti

Componente	File	Ruolo
<code>WSClientController</code>	<code>kitchenmgmt.controller.ts</code>	Connessione STOMP, <code>_pkmiQueue</code> seriale, retry automatico
<code>KitchenManagementController</code>	<code>kitchenmgmt.controller.ts</code>	REST fetch items paginato, fetch singolo item per ID
<code>handleIncomingData</code>	<code>dispatcher.ts</code>	Filtro/normalizzazione messaggi STOMP, salva plate TODO
<code>handleSingleOrderData</code>	<code>dispatcher.ts</code>	Normalizzazione array sync/resync, risolve plate originale
<code>handleIncomingOrder</code>	<code>dispatcher.ts</code>	Routing stampante, idempotency check, accodamento stampa
<code>enqueuePrinterJob</code>	<code>dispatcher.ts</code>	Coda sequenziale per-stampante (Promise chain),

Componente	File	Ruolo
		100ms inter-print delay
<code>scheduleResync</code>	<code>dispatcher.ts</code>	Debounce 3000ms su PKMI_UPDATE PROGRESS/DONE
<code>reconcileMissedOrders</code>	<code>dispatcher.ts</code>	Recovery ordini TODO→DONE persi nel debounce (stampa tardiva)
<code>sendToPrinter</code>	<code>print.ts</code>	Invio buffer ESC/POS via TCP
<code>DatabaseController.saveReceipt</code>	<code>db.controller.ts</code>	Persistenza receipt — fonte per idempotency check
<code>DatabaseController.todo_plate_cache</code>	<code>db.controller.ts</code>	Cache persistente plate originale degli ordini TODO
<code>HttpServerController.sendNotification</code>	<code>httpserver.controller.ts</code>	WebSocket push notifiche ai client UI